

LNS-Madam: Low-Precision Training in Logarithmic Number System Using Multiplicative Weight Update

Jiawei Zhao[✉], Steve Dai[✉], Rangharajan Venkatesan, Brian Zimmer[✉], Mustafa Ali[✉],
Ming-Yu Liu, Brucek Khailany[✉], *Senior Member, IEEE*,
William J. Dally[✉], *Fellow, IEEE*, and Anima Anandkumar[✉], *Fellow, IEEE*

Abstract—Representing deep neural networks (DNNs) in low-precision is a promising approach to enable efficient acceleration and memory reduction. Previous methods that train DNNs in low-precision typically keep a copy of weights in high-precision during the weight updates. Directly training with low-precision weights leads to accuracy degradation due to complex interactions between the low-precision number systems and the learning algorithms. To address this issue, we develop a co-designed low-precision training framework, termed LNS-Madam, in which we jointly design a logarithmic number system (LNS) and a multiplicative weight update algorithm (Madam). We prove that LNS-Madam results in low quantization error during weight updates, leading to stable performance even if the precision is limited. We further propose a hardware design of LNS-Madam that resolves practical challenges in implementing an efficient datapath for LNS computations. Our implementation effectively reduces energy overhead incurred by LNS-to-integer conversion and partial sum accumulation. Experimental results show that LNS-Madam achieves comparable accuracy to full-precision counterparts with only 8 bits on popular computer vision and natural language tasks. Compared to FP32 and FP8, LNS-Madam reduces the energy consumption by over 90% and 55%, respectively.

Index Terms—Machine learning, hardware acceleration, optimization

1 INTRODUCTION

DEEP neural networks (DNNs) have shown impressive performance in many applications, including image classification and language processing. However, training and deploying DNNs typically incurs significant computation and energy costs. Traditionally, values in neural networks are represented using floating-point (32-bit) numbers, which incurs a large arithmetic and memory footprint, and hence significant energy consumption. However, recent studies suggest that high-precision number formats are redundant, and models can be quantized in low-precision with little loss in accuracy [1], [2]. Low-precision numbers only require low-bitwidth computational units, leading to

better computational efficiency and less required memory bandwidth and capacity.

While low-precision training methods generally reduce computational costs, energy efficiency can be further improved by choosing a logarithmic number system (LNS) for representing numbers. LNS achieves a higher computational efficiency by transforming expensive multiplication operations in the network layers to inexpensive additions in their logarithmic representations. In addition, it attains a wide dynamic range and can provide a good approximation of the non-uniform weight distribution in neural networks. Thus LNS is an excellent candidate for training DNNs in low-precision [3], [4], [5].

Although previous studies demonstrate that it is feasible to train networks in low-precision using LNS, these approaches have not yet shown promising results on larger datasets and state-of-the-art models [5], [6]. Standard LNS fixes the base of the logarithm, termed log-base, to be precisely two. However, a more flexible log-base is needed since the numbers in DNNs require different quantization gaps during training [7]. A flexible log-base can introduce additional hardware overhead due to expensive conversion operations between the logarithmic and integer (linear) formats. This motivates us to design a LNS that has a flexible choice of the log-base while maximizing the efficiency of LNS-to-integer conversions.

Conventional low-precision training methods typically require high-precision copies of weights and gradients during weight update to maintain optimization stability. In fact, most recent studies even use a full-precision (FP32) copy of weights [5], [6]. This introduces additional energy

- Jiawei Zhao is with Caltech, Pasadena, CA 91125 USA.
E-mail: jiawei@caltech.edu.
- Steve Dai, Rangharajan Venkatesan, Brian Zimmer, Ming-Yu Liu, Brucek Khailany, and William J. Dally are with NVIDIA, Santa Clara, CA 95050 USA. E-mail: {sdai, rangharajan, bzimmer, mingyu, bkhalany, bdally}@nvidia.com.
- Mustafa Ali is with Purdue University, West Lafayette, IN 47907 USA.
E-mail: ali102@purdue.edu.
- Anima Anandkumar is with Caltech, Pasadena, CA 91125 USA, and also with the NVIDIA, Santa Clara, CA 95050 USA.
E-mail: anima@caltech.edu.

Manuscript received 18 January 2022; revised 12 June 2022; accepted 3 July 2022. Date of publication 22 September 2022; date of current version 11 November 2022.

(Corresponding author: Jiawei Zhao.)

Recommended for acceptance by H. Matsutani.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TC.2022.3202747>, provided by the authors.

Digital Object Identifier no. 10.1109/TC.2022.3202747

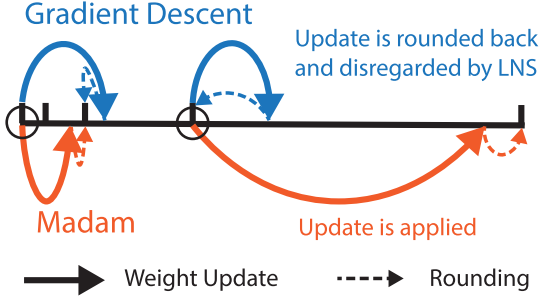


Fig. 1. Illustration for updating weights using Gradient Descent (GD) and Madam under logarithmic representation. Each coordinate represents a number stored in LNS. Assume the weights at two circles receive the same gradient. The updates generated from GD are disregarded as the weights move larger, whereas the updates generated by Madam are adjusted with the weights.

costs and expensive FP32 arithmetic, which becomes prohibitive especially in energy-constrained edge devices.

The high-precision requirement for weight updates is due to complex interactions between learning algorithms and number systems, which has usually been ignored in previous studies. For example, as illustrated in Fig. 1, updates generated by stochastic gradient descent (SGD) are disregarded more frequently by LNS when the weights become larger. This is because the quantization gap grows exponentially as the weights become larger in LNS, which suggests that conventional learning algorithms may not be suitable for LNS. Hence, in previous studies, high-precision weight copies are required to avoid numerical instabilities [5], [7].

To directly update the weights in low-precision, we employ a learning algorithm tailored to LNS. Recently, Bernstein et al. [8] proposed the Madam optimizer based on multiplicative updates, which is equivalent to updating weights additively in the logarithmic space. As illustrated in Fig. 1, Madam generates larger magnitudes of the updates when the weights become larger, making it more suitable for a logarithmic weight representation. However, Bernstein et al. [8] still employ full-precision training with Madam without considering low-precision LNS.

In this work, we propose a co-designed low-precision training framework called LNS-Madam in which we adopt LNS (with a more flexible log-base) for representing DNNs and apply a modified Madam (tailored to LNS) to train these networks. LNS-Madam reduces the precision requirements for all components of the training, including *forward and backward propagation, as well as weight updates*.

Our contributions are summarized as follows:

- 1) We design a multi-base LNS where the log-base can be fractional powers of two. The multi-base LNS accommodates the precision and range requirements of the training dynamics while being hardware-friendly. In addition, we propose an approximation for the addition arithmetic in LNS to further improve its energy efficiency.
- 2) We propose an efficient hardware implementation of LNS-Madam that addresses challenges in designing an efficient datapath for LNS computations, including accumulation and conversion between logarithmic and integer formats. We leverage this implementation to study the energy benefits of training in LNS.

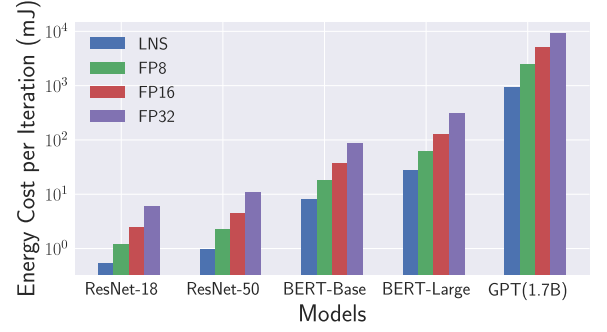


Fig. 2. Energy efficiency for training different models with various number formats. The per-iteration energy consumption (mJ) is listed.

- 3) To achieve low-precision weight updates in LNS, we replace standard SGD or Adam optimizers with our proposed optimizer based on Madam, which directly updates the weights in the LNS. Through theoretical analysis and empirical evaluations, we show that the proposed Madam optimizer achieves significantly lower quantization error in LNS.
- 4) In our experiments, LNS-Madam achieves full-precision accuracy with 8 bit representations on popular computer vision and natural language tasks while reducing energy consumption by over 90%. The energy efficiency results for training different models with various number formats are summarized in Fig. 2.

2 HARDWARE-FRIENDLY MULTI-BASE LOGARITHMIC NUMBER SYSTEM

In this section, we introduce our multi-base logarithmic number system (LNS), including the corresponding number representation and arithmetic operations.

We start with the mathematical formulation that we will use throughout this paper. We assume that the DNN $F(\cdot, W)$ is composed of L layers with learnable weights W and input activations X across the layers. $\mathcal{L}(W)$ denotes the training loss. The forward propagation is defined as: $X_l = f_l(X_{l-1}, W_l)$, where $l \in [1, L]$ denotes layer l . $\nabla_{X_l} = \frac{\partial \mathcal{L}(W)}{\partial X_l}$ and $\nabla_{W_l} = \frac{\partial \mathcal{L}(W)}{\partial W_l}$ denote gradients with respect to input activations and weights, respectively, at layer l . For a number system, we define $\mathcal{B}$ as the bitwidth, x as a number, and x^q as the number in quantized format.

2.1 Multi-Base Logarithmic Representation

Unlike prior work that uses exactly 2 as the base of the logarithmic representation, we propose a multi-base logarithmic representation that allows the base to be two with a fractional exponent, such that

$$x = \text{sign} \times 2^{\tilde{x}/\gamma} \quad \tilde{x} = 0, 1, 2, \dots, 2^{\mathcal{B}-1} - 1,$$

where $\tilde{x}$ is an integer and γ is the base factor that controls the fractional exponent of the base. γ controls the quantization gap, which is the distance between successive representable values within the number system. Previous work has already demonstrated that logarithmic quantized neural networks achieve better performance when relaxing γ from 1 to 2 [5]. We find that further relaxation can help adapt to different models and datasets. Therefore we generalize the

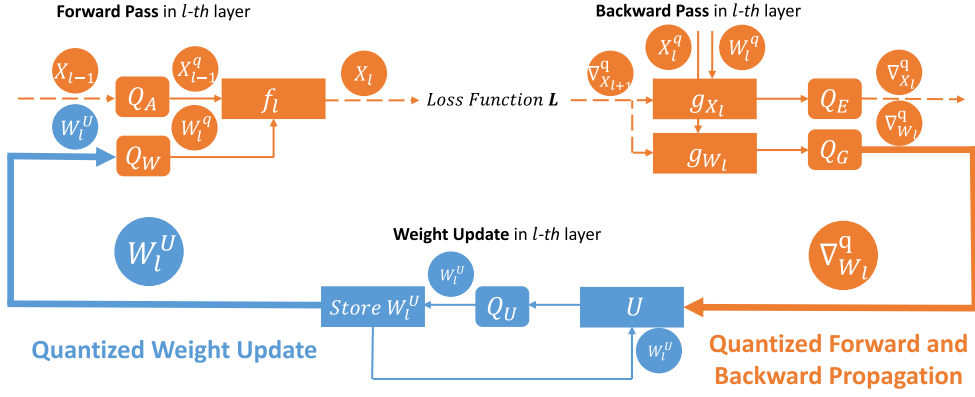


Fig. 3. Illustration of LNS-Madam. Quantized training includes quantizing weights W and activations X in forward propagation, and weight gradients ∇_W and activation gradients ∇_X in backward propagation. g_X and g_W denote the functions to compute gradients. Quantized weight update applies a quantization function Q_U over weights after any learning algorithm U updates them. The quantized weights W^U are the actual numbers stored in the system.

base factor setting, enabling more flexibility in controlling the quantization gap in order to more accurately approximate the training dynamics. In addition, we specially restrict γ to be powers of 2 for hardware efficiency, as described later.

2.2 Arithmetic Operations in LNS

One of the benefits of using LNS stems from the low computational cost of its arithmetic operations. We use dot product operations as an example since they are prevalent during training. Consider two vectors $\mathbf{a} \in \mathbb{R}^n$ and $\mathbf{b} \in \mathbb{R}^n$ that are represented by their integer exponents $\tilde{\mathbf{a}}$ and $\tilde{\mathbf{b}}$ in LNS. A dot product operation between them can be represented as follows:

$$\begin{aligned} \mathbf{a}^T \mathbf{b} &= \sum_{i=1}^n \text{sign}_i \times 2^{\tilde{\mathbf{a}}_i/\gamma} \times 2^{\tilde{\mathbf{b}}_i/\gamma} \\ &= \sum_{i=1}^n \text{sign}_i \times 2^{(\tilde{\mathbf{a}}_i + \tilde{\mathbf{b}}_i)/\gamma} \\ &= \sum_{i=1}^n \text{sign}_i \times 2^{\tilde{\mathbf{p}}_i/\gamma}, \end{aligned} \quad (1)$$

where $\text{sign}_i = \text{sign}(\mathbf{a}_i) \oplus \text{sign}(\mathbf{b}_i)$. In this dot product operation, each element-wise multiplication is computed as an addition between integer exponents, which significantly reduces the computational cost by requiring adders instead of multipliers.

While the multiplications are easy to compute in LNS, the accumulation is difficult to compute efficiently as it requires first converting from logarithmic to integer format and then performing the addition operation. The conversion between these formats is generally expensive as it requires computing $2^{\tilde{\mathbf{p}}_i/\gamma}$ using polynomial expansion. To overcome this challenge, we decompose the exponent $2^{\tilde{\mathbf{p}}_i/\gamma}$ into a quotient component $\tilde{\mathbf{p}}_{iq}$ and a remainder component $\tilde{\mathbf{p}}_{ir}$ as follows:

$$2^{\tilde{\mathbf{p}}_i/\gamma} = 2^{\tilde{\mathbf{p}}_{iq} + \tilde{\mathbf{p}}_{ir}/\gamma} = 2^{\tilde{\mathbf{p}}_{iq}} \cdot 2^{\tilde{\mathbf{p}}_{ir}/\gamma}. \quad (2)$$

With this decomposition, converting from LNS to integer requires only a table lookup for $2^{\tilde{\mathbf{p}}_{ir}/\gamma}$ followed by a shift for $2^{\tilde{\mathbf{p}}_{iq}}$. For the table lookup, we simply maintain γ constants $2^{i/\gamma} \forall i \in \{0, 1, \dots, \gamma - 1\}$ and select the constant based on the remainder $\tilde{\mathbf{p}}_{ir}$. Note that because γ is

restricted to be power of 2, the remainder can be efficiently extracted from the least-significant bits (LSB) of the exponent while the quotient can be extracted from the most-significant bits (MSB). Typically the lookup table (LUT) requires 2^B entries for storing all possible values.

2.3 Conversion Approximation

In addition to the exact conversion technique discussed above, we can further reduce the cost of the LNS-to-integer conversion using a hybrid approximation method. Our method is based on Mitchell approximation [9]: $2^{\tilde{x}/\gamma} \approx (1 + \tilde{x}/\gamma)$, where the logarithmic format can be efficiently approximated to the integer format when $\tilde{x}/\gamma$ is small. Specifically, we further split the remainder into a LSB and a MSB component. The value of the LSB is approximated using Mitchell approximation, and the value of the MSB is performed with table lookup. This helps reduce the size of the LUT. We present a detailed description of our approximation method in the Appendix B, available online. In addition, since the approximation serves as an additional non-linear operation in neural networks, we find the approximated training does not damage accuracy in practice.

3 QUANTIZED FORWARD AND BACKWARD PROPAGATION ON LNS

In this section, we introduce how to apply multi-base LNS to quantized training, as illustrated in Fig. 3.

To realize reduced precision for values and arithmetic during training, we define a logarithmic quantization function $Q_{\log} : \mathbb{R} \rightarrow \mathbb{R}$, which quantizes a real number into a sign and an integer exponent using a limited number of bits. $Q_{\log}$ is defined as follows:

$$Q_{\log}(x) = \text{sign}(x) \times s \times 2^{(\tilde{x}/\gamma)}, \quad (3)$$

where $\tilde{x} = \text{clamp}(\text{round}(\log_2(|x|/s) \times \gamma), 0, 2^{B-1} - 1)$, and $s \in \mathbb{R}$ denotes a scale factor. $Q_{\log}$ first brings scaled numbers $|x|/s$ into their logarithmic space, magnifies them by the base factor γ and then performs rounding and clamping functions to convert them into desired integer exponents $\tilde{x}$.

The scale factor s usually is shared within a group of numbers, and its value is assigned to match the maximum number within the group.

We apply quantization-aware training (QAT) for quantizing weights and activations during forward propagation. Each quantizer is associated with a STE to allow the gradients to directly flow through the non-differentiable quantizer during backward pass [10]. Because QAT views each quantization function as an additional non-linear operation in the networks, the deterministic quantization error introduced by any quantizer in the forward pass is implicitly reduced through training. We define weight quantization function as QW and activation quantization function as QA for each layer during forward propagation, where $W_l^q = QW(W_l)$ and $X_l^q = QA(f_l(X_{l-1}^q, W_l^q))$.

In order to accelerate training in addition to inference, gradients also need to be quantized into low-precision numbers. As shown by recent studies, the distribution of gradients resembles a Gaussian or Log-Normal distribution [11], [12]. This suggests that logarithmic representation may be more suitable than fixed-point representations when quantizing gradients to attain hardware efficiency. We quantize the activation gradients using quantization function QE : $\nabla_{X_l}^q = QE(\nabla_{X_l})$. We also quantize the weight gradients using quantization function QG : $\nabla_{W_l}^q = QG(\nabla_{W_l})$. In this work, we aim to reduce the precision requirement for both weight gradients and activation gradients in the backward pass.

4 MULTIPLICATIVE WEIGHT UPDATE ALGORITHM FOR LNS

Although logarithmic quantized training significantly improves training efficiency, its overall efficiency continues to be hampered by the high precision requirement of weight updates. We note that quantized weight update is orthogonal to quantized training due to the difference in their objectives. Quantized training tries to maintain the fidelity of weight gradients while accelerating forward and backward propagation. This provides accurate gradient information for the weight update. On the other hand, after receiving quantized weight gradients, quantized weight update aims to reduce gaps between updated weights and their (rounded) quantized counterparts. Fig. 3 distinguishes the two parts by different colors.

Previous works generally assume that the weight update is computed over a full-precision weight space. In other words, a full-precision copy of weights is maintained [5], [7] and very little rounding follows weight update. However, this offsets the efficiency benefits of quantized training and requires expensive floating-point arithmetic not available especially in cheap energy-constrained edge devices. Therefore, in this work, we consider quantized weight update in LNS, where the weights are updated over a discrete logarithmic space instead of a full-precision one. We aim to minimize the rounding error given that weights are represented in LNS.

4.1 Quantized Weight Update

To better understand this problem, we first define a generalized form of a weight update as: $W_{t+1} = U(W_t, \nabla_{W_t})$, where

U represents any learning algorithm. For example, gradient descent (GD) algorithm takes $U_{GD} = W_t - \eta \nabla_{W_t}$, where η is learning rate.

Because the weights need to be represented in a quantized format in LNS, it is necessary to consider the effect of logarithmic quantization during weight update. We define *logarithmic quantized weight update* as follows:

$$W_{t+1}^U = Q\log(U(W_t, \nabla_{W_t})). \quad (4)$$

In this case, W_{t+1}^U can be directly stored in a logarithmic format without using floating-point data type. For simplicity, we assume weight gradients ∇_{W_t} are exact as quantized training is orthogonal to this problem. Switching to the approximated gradient estimates will not affect our theoretical results.

4.2 Quantization Error Analysis

Because the logarithmic quantization requires representing values in a discrete logarithmic scale, quantized weight update inevitably introduces a mismatch between the quantized weights and their full-precision counterparts. To preserve the reliability of the optimization, we aim to reduce the quantization error (i.e., the mismatch). For the following, we take a theoretical perspective to discuss how different learning algorithms affect the quantization error under LNS. Detailed assumptions and proofs can be found in Appendix A, available in the online supplemental material.

Due to the logarithmic structure, we focus on minimizing a quantization error $r_t = \|\log_2 |W_{t+1}^U| - \log_2 |W_{t+1}|\|^2$, which measures the L2 norm of the difference between the weights and their quantized counterparts in logarithmic space. Because r_t quantify the relative difference between $|W_{t+1}^U|$ and $|W_{t+1}|$, minimizing r_t is largely similar to minimizing the relative quantization error $\|(W_{t+1} - W_{t+1}^U)/W_{t+1}\|^2$.

We assume a simplified logarithmic quantization where the scale factor and the clamping function are ignored. This ensures our focus is on the effect of the quantization gap determined by γ instead of the dynamic range. We also replace the deterministic rounding with a stochastic counterpart SR where $\mathbb{E}SR(x) = x$ for any real number. Although SR helps us establish the theoretical results, in practice SR requires random generators that induce additional costs, and thus are not suitable for energy-efficient training.

Everything we need, we use gradient descent as an example to discuss why traditional learning algorithms are not suited for LNS-based quantized weight updates. The theorem is stated as follows:

Theorem 1. *The quantization error $r_{t,GD}$ introduced by logarithmic quantized gradient descent at iteration t can be bounded in expectation, as*

$$\mathbb{E}r_{t,GD} \leq \frac{\sqrt{d}}{\gamma} \|\log_2(|W_t| - \eta_1 \nabla_{W_t})\|, \quad (5)$$

where d is the dimension of W and η_1 is the learning rate of U_{GD} .

Theorem 1 suggests that $r_{t,GD}$ is magnified when the magnitudes of weights become larger. This is because the updates $\eta_1 \nabla_{W_t}$ generated by GD are not proportional to the

magnitudes of weights. $\eta_1 \nabla_{W_t}$ can be orders of magnitude smaller than the quantization gaps as weights become larger, and thus these updates often are disregarded by quantization function $Q_{\log}$. We intuitively illustrate this problem in Fig. 1.

To ensure the updates are proportional to the weights, a straightforward way is to update the weights multiplicatively. Because the weights are represented in LNS, we further consider a special multiplicative learning algorithm tailored to LNS, which updates the weights directly over their logarithmic space

$$U_{MUL} = \text{sign}(W_t) \odot 2^{\tilde{W}_t - \eta \nabla_{W_t} \odot \text{sign}(W_t)} \quad (6)$$

where $\tilde{W}_t = \log_2 |W_t|$ are the exponents of the magnitude of weights, and $\odot$ denotes element-wise multiplication. U_{MUL} makes sure the magnitude of each element $W_t(k)$ of the weights decreases when the sign $\text{sign}(W_t(k))$ and $\nabla_{W_t(k)}$ agree and increases otherwise. The quantization error with regards to U_{MUL} is stated as follows:

Theorem 2. *The quantization error $r_{t,MUL}$ introduced by logarithmic quantized multiplicative weight update at iteration t can be bounded in expectation, as*

$$\mathbb{E}r_{t,MUL} \leq \frac{\sqrt{d} \eta_2}{\gamma} \|\nabla_{W_t}\|, \quad (7)$$

where d is the dimension of W and η_2 is the learning rate of U_{MUL} .

Theorem 2 indicates that $r_{t,MUL}$ does not depend on the magnitudes of weights, and thus the quantization error is not magnified when the weights become larger. This is in stark contrast to the quantization error from gradient descent shown in Equation (5). The comparison is illustrated in Fig. 1.

Interestingly, we find that the quantization error $r_{t,MUL}$ can be further simplified by regularizing the information of gradients for the learning algorithm U_{MUL} :

Lemma 1. *Assume the multiplicative learning algorithm U_{MUL} only receives the sign information of gradients where $U_{MUL} = \tilde{W}_t - \eta_2 \text{sign}(\nabla_{W_t}) \odot \text{sign}(W_t)$. The upper bound on quantization error $r_{t,MUL}$ becomes*

$$\mathbb{E}r_{t,MUL} \leq \frac{d \eta_2}{\gamma}. \quad (8)$$

The result in Lemma 1 suggests that $r_{t,MUL}$ can be independent of both weights and gradients when only taking sign information of gradients during weight update. We denote this special learning algorithm as U_{signMUL} . U_{signMUL} is a multiplicative version of signSGD, which has been studied widely [12], [13].

To verify our theoretical results, we empirically measure the quantization errors for the three aforementioned learning algorithms over a range of η and γ . As shown in Fig. 4, the empirical findings are in line with our theoretical results. Although all learning algorithms introduce less errors when η and γ become smaller, the multiplicative algorithms introduce significantly lower errors compared to the gradient descent.

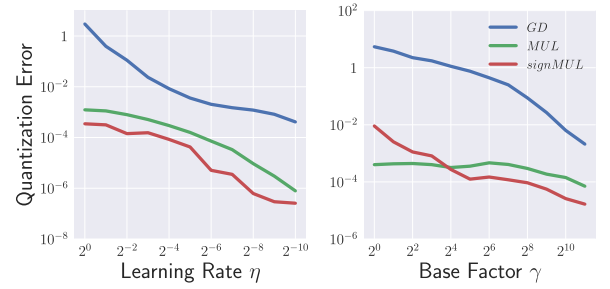


Fig. 4. Quantization error from different learning algorithms on ImageNet. The errors are averaged over all iterations in the first epoch. The results suggest that multiplicative algorithms introduce significantly lower errors compared to the gradient descent, which are also in line with our theoretical results.

In addition to reducing the quantization error in the quantized weight update, U_{signMUL} must also have the ability to minimize the loss function $\mathcal{L}(W)$. Interestingly, we notice that U_{signMUL} resembles a recently proposed learning algorithm Madam, where Bernstein et al. [8] proves that Madam optimizes the weights in a descent direction. Madam updates the weights multiplicatively using normalized gradients

$$U_{\text{Madam}} = W_t \odot e^{-\eta \text{sign}(W_t) \odot g_t^*} \quad (9)$$

$$g_t^* = g_t / \sqrt{g_2}$$

where g_t represents the gradient vector ∇_{W_t} , and g_t^* denote a normalized gradient, which is the fraction between g_t and the square root of its second moment estimate $\sqrt{g_2}$. Bernstein et al. [8] demonstrates that Madam achieves state-of-the-art accuracy over multiple tasks with a relatively fixed learning rate η . They also theoretically prove the descent property of Madam that ensures its convergence. Although Bernstein et al. [8] further shows the possibility of applying Madam over a discrete logarithmic weight space, they still employ full-precision training without considering low-precision LNS.

Algorithm 1. Madam Optimizer on LNS

Require: Base-2 weight exponents $\tilde{W}$, where $\tilde{W} = \log_2(W)$, learning rate η , momentum β
 Initialize $g_2 \leftarrow 0$
repeat
 $g \leftarrow \text{StochasticGradient}()$
 $g_2 \leftarrow (1 - \beta)g^2 + \beta g_2$
 $g^* \leftarrow g / \sqrt{g_2}$
 $\tilde{W} \leftarrow \tilde{W} - \eta g^* \odot \text{sign}(W)$
until converged

To ensure low-precision weight updates in LNS, we apply a modified version of the Madam optimizer to enable fast convergence while preserving low quantization error. The modified Madam directly optimizes the weights over their base-2 logarithmic space using the gradient normalization technique described in Equation (9). Details of our optimizer are shown in Algorithm 1. Because our Madam optimizer directly updates base-2 exponents of weights in LNS, there is no need for integer-to-LNS conversion during weight update when the weights are already in LNS, further reducing the energy cost.

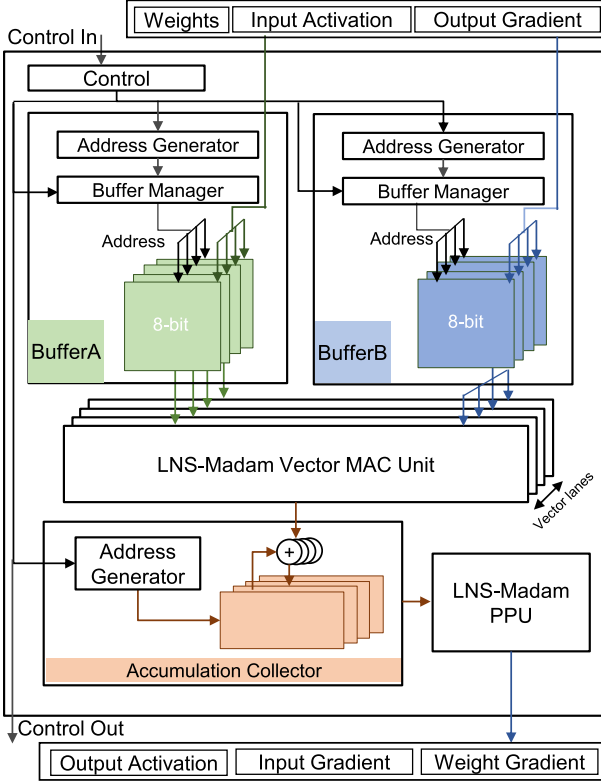


Fig. 5. LNS-Madam processing element (PE).

5 HARDWARE IMPLEMENTATION

We extend a previously optimized DNN accelerator [14] to support LNS-based DNN computations. Fig. 5 shows the micro-architecture of the PE which performs dot-product operations. Each PE consists of set of vector MAC units fed by the buffers that store weights, input activations, and output gradients. Additionally, the accumulation collectors store and accumulate partial sums which are passed to the PPU for post-processing (e.g., quantization scaling, non-linear activation functions) if necessary.

Fig. 6 shows the LNS-based datapath inside the LNS-Madam Vector MAC Unit. Here we model exact LNS-to-integer conversion without any approximation. With a vector size of 32 and input bitwidths of 8, the datapath

processes 32 7-bit exponent values at each of its exponent inputs (e_a and e_b) and 32 1-bit sign values (s_a and s_b) at each of its sign inputs to produce a 24-bit partial sum. First, the LNS datapath performs the dot-product multiplication by adding the exponents and XOR-ing the sign bits. The output of the product computation requires an additional bit to account for the carry-out of the adder. At this point, each exponent is split into a quotient component (e_q) and a remainder component (e_r) based on the LSB/MSB property mentioned in Section 2.2. Second, the datapath performs shifting by the quotient to implement the quotient component in Equation (2). Depending on the sign bit, the corresponding signed shifted value is selected and passed to the corresponding adder tree based on the remainder select signal. Third, the result of shifted values are reduced through the set of adder trees and registered. At last, the results of the adder trees are multiplied with corresponding remainder constants (described in Section 2.2) from a LUT and accumulated into the final partial sum, represented in integer (linear) format. This partial sum needs to be converted back into logarithmic format and written back to the global buffer for subsequent LNS-based computations.

Additional microarchitectural details of the PE are listed in Table 1. Notably, our accelerator uses a multi-level data-flow called output-stationary local-A-stationary[14] to optimize reuse across different operands. Inputs from buffer A are read out once every 16 cycles and stored in a register for temporal reuse. Inputs from buffer B are read once every cycle and reused across the 32 lanes spatially. Partial sums are temporally accumulated in a 16-entry latch array collector before sending the completed sum to the post-processing unit. The two buffers in the PE store different data depending on whether output activation, input gradient, or weight gradient is being computed. For example, weights and input activations are stored in BufferA and BufferB respectively during forward propagation to compute the output activations. On the other hand, input activations and output gradients are stored in the respective buffers during backward propagation to compute the weight gradient. Table 2 outlines how we map various tensors in DNN computation to buffers in our hardware during different computation passes. Note that weight updates are performed outside of the PEs through the global buffer.

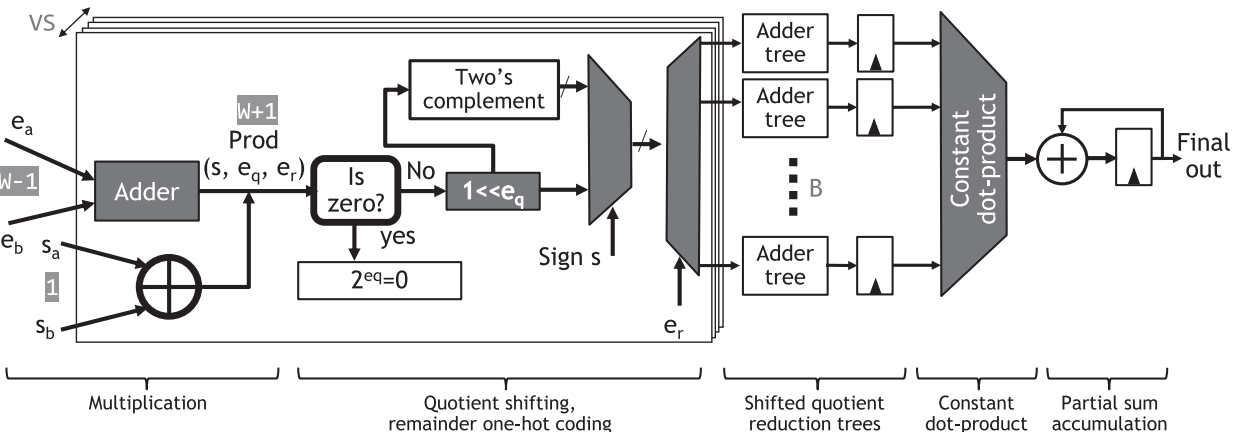


Fig. 6. LNS-Madam Vector MAC Unit – Performs dot-products of inputs represented in LNS and produces partial sum outputs in integer format. Bitwidths of different signals are highlighted. VS stands for vector size; w stands for bitwidth of input values; B refers to base factor and number of remainder bins.

TABLE 1
Microarchitectural Details of LNS-Madam PE

Dataflow	Multi-level
Vector size / # Vector lanes	32
Weight/activation precision	8-bit
Gradient precision	8-bit
# Remainder Bins	8
Accumulation precision	24-bit
Accumulation collector size	1.5 KB
BufferA size	128 KB
BufferB size	8 KB

6 EXPERIMENTS

In this section, we evaluate both the accuracy and energy efficiency of using LNS-Madam to train state-of-the-art models on large-scale datasets.

6.1 Model Accuracy

To evaluate accuracy, we simulate LNS-Madam using a PyTorch-based neural network quantization library that implements a set of common neural network layers (e.g., convolution, fully-connected) for training and inference in both full and quantized modes [15]. The baseline library supports integer quantization in a fixed-point number system, and we further extend it to support LNS. The library also provides utilities for scaling values to the representable integer range of the specific number format. With this library, a typical quantized layer consists of a conventional layer implemented in floating-point preceded by a weight quantizer and an input quantizer that converts the weights and inputs of the layer to the desired quantized format. For the backward pass, after the gradients pass through the STE in each quantizer, they will be quantized by their quantizers as well.

We benchmark LNS-Madam on various tasks including ResNet models on CIFAR-10 and ImageNet, and BERT-base and BERT-large language models on SQuAD and GLUE. Specifically, we train ResNet models from scratch on CIFAR-10 and ImageNet, and fine-tune pre-trained BERT models on SQuAD and GLUE. Detailed descriptions of datasets and models can be found in the appendix, available in the online supplemental material.

6.1.1 Parameter Settings

We fix the bitwidth to be 8-bit for both forward and backward propagation, which includes the bitwidth for weights, activations, activation gradients, and weight gradients. We note that 8-bit weight gradients are lower than previous studies that use 16-bit or even 32-bit weight gradients [5], [6].

TABLE 2
Mapping of Tensors to Buffers in PE During Different Computation Passes¹

Pass	BufferA	BufferB
Forward	Weight	Input Activation
Backward (Input)	Weight	Output Gradient
Backward (Weight)	Input Activation	Output Gradient

¹Backward pass consists of backward computation for input gradient, denoted *Backward(Input)*, and backward computation for weight gradient, denoted *Backward(Weight)*.

TABLE 3
Base Factor Selection on ImageNet^{1,2}

γ	Dynamic Range	Forward	Backward
1	(0,127)	NaN	NaN
2	(0,63.5)	75.81	75.79
4	(0,31.8)	75.96	76.07
8	(0,15.9)	75.88	76.23
16	(0,7.9)	76.32	63.67
32	(0,4.0)	68.15	20.71

¹Bitwidth is 8-bit across settings. Quant Forward or Quant Backward denotes the settings where either forward propagation or backward propagation is quantized while leaving the rest of computation in full-precision.

²The results of test accuracy (%) are listed.

To find an appropriate base factor γ under the 8-bit setting, we vary γ to find the appropriate dynamic ranges for forward and backward quantization. The dynamic range in LNS is $(0, (2^{B-1} - 1)/\gamma)$, which is controlled by both bitwidth and base factor.

As shown in Table 3, we fix the bitwidth as 8-bit and vary the base factor γ to find the appropriate dynamic ranges for forward and backward quantization. According to the results, we find the base factor of 8 with the dynamic range (0,15.9) that uniformly works across QW, QA, QE, and QG.

In order to maintain optimization stability, the bitwidth of the weight updates require to be larger than the bitwidth of the weights. When the bitwidth of QU is larger than 8-bit, we increase its base factor to match the desired dynamic range (0,15.9).

We empirically search the best learning rate η for our Madam optimizer from 2^{-4} to 2^{-10} , and we find $\eta = 2^{-7}$ works best uniformly across tasks, which suggests the learning rate for Madam is robust.

6.1.2 Comparisons

Given the settings above, we compare LNS-Madam with FP8 and FP32. For FP8 and FP32, the standard optimizers are applied for tasks by default. We use a tuned SGD optimizer for CIFAR-10 and ImageNet datasets, and a tuned AdamW optimizer for SQuAD and GLUE datasets. For all settings, we use per-channel scaling for ResNet and per-feature scaling for BERT, both of which are commonly used scaling techniques. The clamping function is performed by matching the largest value within each group of numbers.

To demonstrate LNS-Madam is better than popular number systems, we compare it with both FP8 and FP32, where our FP8 contains 4-bit exponent and 3-bit mantissa. As shown in Table 4, LNS-Madam yields better performance than FP8, and it even achieves performance comparable to the full-precision counterpart.

In addition, we compare LNS-Madam with recent methods on low-precision training. For all methods, we fix forward and backward propagation in 8-bit while varying the weight update precision from 32-bit to 16-bit. As shown in Table 5, LNS-Madam achieves the best accuracy under 16-bit weight update, which demonstrates its effectiveness under the quantized setting. FP8 [2] also achieves negligible degradation after switching to 16-bit, as it applies stochastic rounding over weight update process. Since BHQ [16] achieves the best

TABLE 4
Benchmarking LNS-Madam on Various Datasets and Models¹

Dataset	Model	LNS-Madam ²	FP8 ²	FP32
CIFAR-10	ResNet-18	93.41	93.12	93.51
ImageNet	ResNet-50	76.14	75.83	76.38
SQuAD	BERT-base	88.13	88.07	88.36
SQuAD	BERT-large	90.75	90.54	90.80
GLUE	BERT-base	88.89	88.73	88.92
GLUE	BERT-large	89.24	88.91	89.35

¹The results of test accuracy (%) are listed.

²Forward and backward propagation are in 8-bit, and the weight update is in 16-bit.

TABLE 5
Comparing LNS-Madam With Recent Low-Precision Training Methods on 8-Bit Training¹

	Data format	16-bit	32-bit
LNS-Madam	LNS	76.14	76.23
BHQ [16]	INT	74.89	76.35
Unified INT8 [17]	INT	74.73	76.27
FP8 [2]	FP	71.46	71.53

¹Evaluate ResNet-50 on ImageNet. Forward and backward propagation are in 8-bit. Test accuracy (%) evaluated under 32-bit and 16-bit weight update are presented.

TABLE 6
Comparing LNS-Madam and BHQ Over a Range of Bitwidth¹

	4-bit	5-bit	6-bit	7-bit	8-bit
LNS-Madam	74.23	75.89	74.41	76.16	76.23
BHQ [16]	74.04	75.70	76.21	76.14	76.35

¹Bitwidth of activation gradients varies from 4-bit to 8-bit. The results of test accuracy (%) are listed.

accuracy under 32-bit, we also compare it with LNS-Madam over a range of bitwidth settings in Table 6.

We also compare Madam with the default optimizers SGD and AdamW under logarithmic quantized weight update, as defined in Equation (4). All optimizers use the same learning rates as above.

As shown in Fig. 7, we vary the bitwidth of the quantized weight update Q_U from 16-bit to 10-bit to test their performance over a wide range. The results suggest compared to other optimizers, *Madam always maintains higher accuracy when precision is severely limited*. Notably, for BERT model on SQuAD and GLUE benchmarks, Madam is 20% better than Adam with respect to F-1 score, when the weight update is in 10-bit. We observe large degradation for both Madam and SGD on ImageNet training, and we believe this is

TABLE 7
Design Tools Used for LNS-Madam Hardware Experiments

HLS Compiler	Mentor Graphics Catapult HLS
Verilog simulator	Synopsys VCS
Logic synthesis	Synopsys Design Compiler
Place-and-route	Synopsys ICC2
Power Analysis	Synopsys PT-PX

because the weights in some layers inevitably require higher precision settings. We leave it as future work to explore LNS-Madam under a customized precision setting.

6.2 Energy Efficiency

We leverage the hardware implementation described in Section 5 to evaluate the energy efficiency of LNS-Madam. We code the hardware model in C++ and Verilog and synthesize it to a combined cycle-accurate RTL using a commercial high-level synthesis tool [18]. Once the RTL is generated, a standard logic synthesis flow is used to obtain the gate-level netlist that is then simulated with representative inputs. To extract energy consumption, we supply the gate-level simulation results from post-synthesis to a standard power analysis tool. We then use an analytical model to compute the total energy consumption for different workloads. We perform our analysis in a sub-16nm state-of-the-art process technology at 0.6V targeting a frequency of 1.05 GHz. Table 7 summarizes the design tools used in the evaluation.

In our experiment, the LNS-Madam hardware is designed with bitwidth $B = 8$ and base factor $\gamma = 8$ for both forward and backward computations. In addition to experimenting with the LNS-based datapath shown in Fig. 6, we also consider FP8, FP16, and FP32 datapath baselines for comparison.

Table 8 presents the energy efficiency per iteration of one forward pass and one backward pass of training. Because different number systems share the same training iterations, per-iteration energy results imply the energy comparison over the entire training. We also present the energy breakdown of the whole PE in Fig. 8. As shown in the figure, FP arithmetic is extremely expensive, contributing a large fraction to the total energy consumption of PEs. The proposed LNS datapath offers significant reduction in the logic complexity, leading to 2.2X, 4.6X, and 11X energy efficiency improvements over FP8, FP16, and FP32 implementations, respectively. We also provide a detailed energy breakdown showing different components of the LNS PE in Fig. 9. In addition, Fig. 10 shows the energy efficiency over a range of GPT models from 1 billion to 1 trillion parameters.



Fig. 7. Comparing Madam with SGD and Adam optimizers under the logarithmic quantized weight update (defined in Equation (4)). The bitwidth of the weight update Q_U is varied from 16-bit to 10-bit.

TABLE 8
Energy Efficiency for Different Models and Number Formats¹

Model	LNS	FP8	FP16	FP32
ResNet-18	0.54	1.22	2.50	5.99
ResNet-50	0.99	2.25	4.59	11.03
BERT-Base	7.99	18.23	37.21	89.35
BERT-Large	27.85	63.58	129.74	311.58

¹The per-iteration energy consumption in mJ are listed.

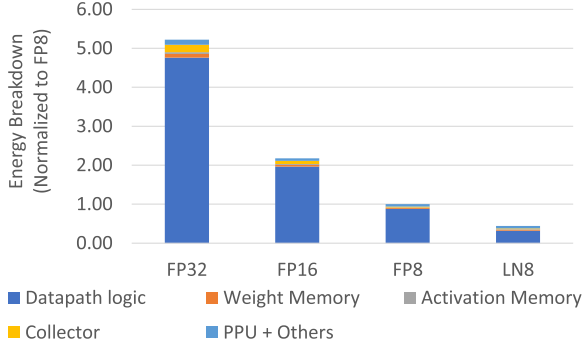


Fig. 8. Energy breakdown of the PE shown in Fig. 6 for different dataformats.

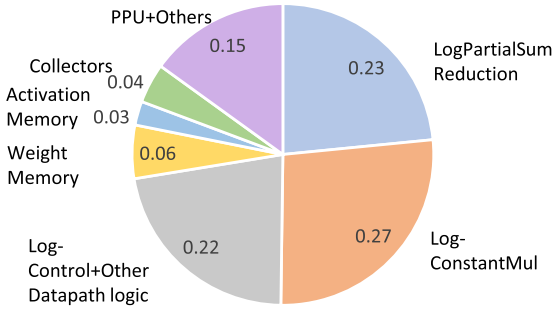


Fig. 9. LNS PE energy breakdown showing different components of datapath

7 RELATED WORKS

7.1 Low-Precision Training

To achieve good accuracy at reduced precision, quantization-aware training (QAT) is commonly applied to directly train the quantized model using straight-through estimators [10], [22], [23], [24], [25]. To accelerate the training phase, several studies suggest quantizing the gradients during backward propagation [2], [6], [26]. To maintain the fidelity of the gradient accumulation, some low-precision training methods assume a full-precision copy for weights during the weight

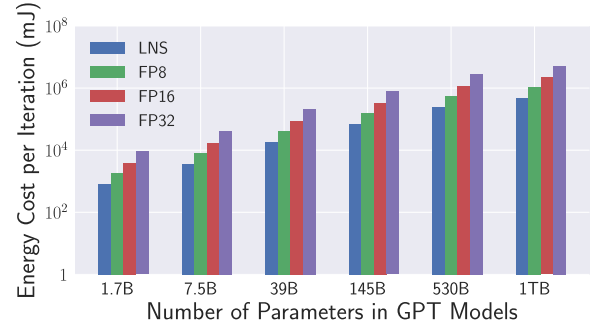


Fig. 10. Energy efficiency over a range of GPT models from 1 billion to 1 trillion parameters. The models are scaled by a throughput efficient method proposed by Narayanan et al. [21].

update [16], [26]. Other studies reduce the precision for the weight update by using high-precision gradient accumulator [27], stochastic rounding [2], [28] or additionally quantizing the residual part of weights [29], [30]. Cambricon-Q accelerates the weight update from a hardware perspective by avoiding costly data transferring in weight update [31]. However, they mostly apply SGD or Adam during the weight update without considering the relationship between the precision of the weights and the underlying learning algorithms.

7.2 Logarithmic Number System

Previous works demonstrate the effectiveness of using logarithmic representation for DNNs [5], [20], [32], [33]. Furthermore, some studies suggest using multiple levels of log-base to reduce the quantization error [5], [7]. However, few of them address the additional computational cost induced by this multi-base design nor scale the system to state-of-the-art neural networks for both training and inference. From the perspective of hardware design, a few studies focus on improving the efficiency of LNS by utilizing the significant cost reduction of multiplications [32], [33], [34], [35], [36]. We compare LNS-Madam with recent LNS-based designs on Table 9.

7.3 Multiplicative Weight Update

Multiplicative algorithms, such as exponentiated gradient algorithm and Hedge algorithm in AdaBoost framework [37], [38], have been well studied in the field of machine learning. In general, multiplicative updates are applied to problems where the optimization domain's geometry is described by relative entropy, such as probability distribution [37]. Recently, [8] proposes an optimizer Madam that focuses on optimization domains described by any relative distance measure instead of only relative entropy. Madam shows great

TABLE 9
Comparing LNS-Madam With Recent LNS-Based Designs

	LNS-Madam	Arnab et al. [19]	Miyashita et al. [5]	Lee et al. [20]	Vogel et al. [7]
Support inference or fine-tuning?	✓	✓	✓	✓	✓
Support training from scratch?	✓	✓	✗	✗	✗
Weight update precision	<16-bit	32-bit	32-bit	32-bit	32-bit
Efficient log-to-linear conversion support	✓	✓	✓	✓	✗
Large-scale evaluation	✓	✗	✓	✓	✓

performance in training large-scale neural networks. However, Madam requires full-precision training without considering its connection to LNS-based low-precision training.

8 CONCLUSION

In this work, we propose a co-designed low-precision training framework LNS-Madam that jointly considers the logarithmic number system and the multiplicative weight update algorithm. Experimental results show that LNS-Madam achieves comparable accuracy to full-precision counterparts even when forward and backward propagation, and weight updates are all in low-precision. To support the training framework in practice, we design a hardware implementation of LNS-Madam to efficiently perform the necessary LNS computations for DNN training. Based on our energy analysis, LNS-Madam reduces energy consumption by over 90% compared to a floating-point baseline.

An important application of our low-precision training framework is learning neural networks over energy-constrained edge devices. This is fundamental for intelligent edge devices to easily adapt to changing and non-stationary environments by learning on-device and on-the-fly. By enabling highly energy-efficient training, our work carries the promising opportunity for using LNS-based hardware to conduct environmental-friendly deep learning research in the near future.

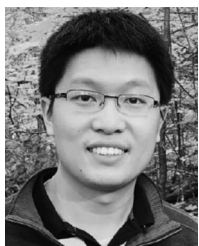
REFERENCES

- [1] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 1737–1746. [Online]. Available: <http://proceedings.mlr.press/v37/gupta15.html>
- [2] N. Wang, J. Choi, D. Brand, C. Chen, and K. Gopalakrishnan, "Training deep neural networks with 8-bit floating point numbers," in *Proc. Adv. Neural Informat. Process. Syst. 31: Annu. Conf. Neural Inf. Process. Syst.*, 2018, pp. 7686–7695. [Online]. Available: <https://proceedings.neurips.cc/paper/2018/hash/335d3d1cd7ef05ec77714a215134914c-Abstract.html>
- [3] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. 7th Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=rJl-b3RcF7>
- [4] J. Bartol et al., "Nanocconnectomic upper bound on the variability of synaptic plasticity," *eLife*, vol. 4, Nov. 2015, Art. no. e10778.
- [5] D. Miyashita, E. H. Lee, and B. Murmann, "Convolutional neural networks using logarithmic data representation," 2016, *arXiv:1603.01025*.
- [6] X. Sun et al., "Ultra-low precision 4-bit training of deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst. 33: Annu. Conf. Neural Informat. Process. Syst.*, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/13b919438259814cd5be8cb45877d577-Abstract.html>
- [7] S. Vogel, M. Liang, A. Guntoro, W. Stechele, and G. Ascheid, "Efficient hardware acceleration of cnns using logarithmic data representation with arbitrary log-base," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2018, pp. 1–8.
- [8] J. Bernstein, J. Zhao, M. Meister, M. Liu, A. Anandkumar, and Y. Yue, "Learning compositional functions via multiplicative weight updates," in *Proc. Adv. Neural Inf. Process. Syst. 33: Annu. Conf. Neural Inf. Process. Syst.*, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/9a32ef65c42085537062753ec435750f-Abstract.html>
- [9] J. N. Mitchell, "Computer multiplication and division using binary logarithms," *IRE Trans. Electron. Comput.*, vol. EC-11, no. 4, pp. 512–517, 1962.
- [10] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
- [11] B. Chmiel, L. Ben-Uri, M. Shkolnik, E. Hoffer, R. Banner, and D. Soudry, "Neural gradients are near-lognormal: Improved quantized and sparse training," in *Proc. 9th Int. Conf. Learn. Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=EoFNy62JGd>
- [12] J. Bernstein, J. Zhao, K. Azizzadenesheli, and A. Anandkumar, "signSGD with majority vote is communication efficient and fault tolerant," in *Proc. 7th Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=BJxhijAcY7>
- [13] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 560–569. [Online]. Available: <https://proceedings.mlr.press/v80/bernstein18a.html>
- [14] R. Venkatesan et al., "MAGNet: A modular accelerator generator for neural networks," *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2019, pp. 1–8.
- [15] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Mickevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," 2020, *arXiv:2004.09602*. [Online]. Available: <https://arxiv.org/abs/2004.09602>
- [16] J. Chen, Y. Gai, Z. Yao, M. W. Mahoney, and J. E. Gonzalez, "A statistical framework for low-bitwidth training of deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst. 33: Annu. Conf. Neural Inf. Process. Syst.*, 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/099fe6b0b444c23836c4a5d07346082b-Abstract.html>
- [17] F. Zhu et al., "Towards unified INT8 training for convolutional neural network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1969–1976.
- [18] M. C. McFarland, A. C. Parker, and R. Camposano, "The high-level synthesis of digital systems," *Proc. IEEE*, vol. 78, no. 2, pp. 301–318, Feb. 1990.
- [19] A. Sanyal, P. A. Beerel, and K. M. Chugg, "Neural network training with approximate logarithmic computations," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2020, pp. 3122–3126.
- [20] E. H. Lee, D. Miyashita, E. Chai, B. Murmann, and S. S. Wong, "LogNet: Energy-efficient neural networks using logarithmic computation," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2017, pp. 5900–5904.
- [21] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2021.
- [22] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients," 2016, *arXiv:1606.06160*. [Online]. Available: <https://arxiv.org/abs/1606.06160>
- [23] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: Imagenet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 525–542.
- [24] A. Zhou, A. Yao, Y. Guo, L. Xu, and Y. Chen, "Incremental network quantization: Towards lossless CNNs with low-precision weights," in *Proc. 5th Int. Conf. Learn. Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=HyQJ-mclg>
- [25] B. Jacob et al., "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2704–2713. [Online]. Available: http://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [26] R. Banner, I. Hubara, E. Hoffer, and D. Soudry, "Scalable methods for 8-bit training of neural networks," in *Proc. Adv. Neural Inf. Process. Syst. 31: Annu. Conf. Neural Inf. Process. Syst.*, 2018, pp. 5151–5159. [Online]. Available: <https://proceedings.neurips.cc/paper/2018/hash/e82c4b19b8151ddc25d4d93baf7b908f-Abstract.html>
- [27] C. Sakr and N. R. Shanbhag, "Per-tensor fixed-point quantization of the back-propagation algorithm," in *Proc. 7th Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=rkxANjA9Ym>
- [28] S. Wu, G. Li, F. Chen, and L. Shi, "Training and inference with integers in deep neural networks," in *Proc. 6th Int. Conf. Learn. Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=HJGXzmspb>
- [29] X. Sun, "Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst. 32: Annu. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4901–4910. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/65fc9fb4897a89789352e211ca2d398f-Abstract.html>

- [30] C. D. Sa et al., "High-accuracy low-precision training," 2018, *arXiv:1803.03383*.
- [31] Y. Zhao et al., "Cambricon-Q: A hybrid architecture for efficient training," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Comput. Architecture.*, 2021, pp. 706–719. [Online]. Available: <https://ieeexplore.ieee.org/document/9499944/>
- [32] J. Johnson, "Rethinking floating point for deep learning," 2018, *arXiv:1811.01721*.
- [33] J. Johnson, "Efficient, arbitrarily high precision hardware logarithmic arithmetic for linear algebra," in *Proc. IEEE 27th Symp. Comput. Arithmetic*, 2020, pp. 25–32.
- [34] H. Saadat, H. Bokhari, and S. Parameswaran, "Minimally biased multipliers for approximate integer and floating-point multiplication," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2623–2635, Nov. 2018.
- [35] H. Saadat, H. Javaid, and S. Parameswaran, "Approximate integer and floating-point dividers with near-zero error bias," in *Proc. ACM/IEEE 56th Des. Automat. Conf.*, 2019, pp. 1–6.
- [36] H. Saadat, H. Javaid, A. Ignjatovic, and S. Parameswaran, "REALM: Reduced-error approximate log-based integer multiplier," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2020, pp. 1366–1371.
- [37] J. Kivinen and M. K. Warmuth, "Exponentiated gradient versus gradient descent for linear predictors," *Inf. Comput.*, vol. 132, no. 1, pp. 1–63, 1997.
- [38] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [39] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [40] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423>
- [41] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.
- [42] J. Deng, W. Dong, R. Socher, L. Li, K. Li, and F. Li, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [43] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2016, pp. 2383–2392. [Online]. Available: <https://aclanthology.org/D16-1264>
- [44] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proc. 7th Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=rj4km2R5t7>



Jiawei Zhao received the BS degree in computer science from NUAU, Nanjing, China, in 2019. He is currently working toward the PhD degree with Caltech, under the supervision of prof. Anima Anandkumar. His research interest lies in machine learning, specifically in deep learning and optimization. He is working on the development of novel and efficient algorithms for solving deep learning problems, such as distributed or low-precision training.



Steve Dai received the BS degree in electrical engineering from the University of California, Los Angeles, in 2011, the MS degree in electrical engineering from Stanford University, in 2013, and the PhD degree in electrical and computer engineering from Cornell University, in 2019. He is currently a research scientist as part of the ASIC & VLSI Research Group, NVIDIA. His research interests include energy-efficient DL acceleration, high-level design methodologies, and ML-assisted EDA.



Rangharajan Venkatesan received the BTech degree in electronics and communication engineering from the Indian Institute of Technology, Roorkee, India, in 2009 and the PhD degree in electrical and computer engineering from Purdue University, West Lafayette, USA, in 2014. He is a senior research scientist with NVIDIA. His research interests include machine learning accelerators, low-power VLSI design, and SoC design methodologies. He has served as a member of the technical program committees of several leading IEEE conferences including International Solid-State Circuits Conference, International Symposium on Micro architecture, Design Automation Conference, and International Symposium on Low Power Electronics and Design.



design, machine learning accelerators, productive design methodologies, and variation tolerance.

Brian Zimmer received the BS degree in electrical engineering from the University of California, Davis, Davis, CA, USA, in 2010, and the MS and PhD degrees in electrical engineering and computer sciences from the University of California at Berkeley, Berkeley, CA, in 2012 and 2015, respectively. He is currently a senior research scientist with the Circuits Research Group, NVIDIA, Inc., Santa Clara, CA. His research interests include soft error resilience, energy-efficient digital design, low-voltage static random-access memory (SRAM)



design, machine learning accelerators, productive design methodologies, and variation tolerance.

Mustafa Ali received the BSc (Hons.) and MSc degrees in electrical engineering from MTC, Cairo, Egypt, in 2011 and 2016, respectively. He is currently working toward the PhD degree with Purdue University, under the guidance of prof. Roy. He was honored the Duty Medal for excellent performance during his studies. He worked on flexible electronics applications using TFTs in his MSc degree from 2014 to 2016. In Addition, he worked as a TA and RA with MTC from 2013 to 2017. He was also a hardware and embedded systems engineer with Integreight, Inc., from 2012 to 2017. He joined the Nano-Electronics Research Lab (NRL), Purdue University, in Spring 2018. His research interests include accelerating brain-inspired computing and machine learning. He is also interested in in-memory computing based on CMOS and post-CMOS devices.



Ming-Yu Liu received the PhD degree in electrical and computer engineering from the Department of Electrical and Computer Engineering, University of Maryland, College Park, Maryland, in 2012. He is currently a distinguished research scientist and a manager with NVIDIA Research, Santa Clara, CA, USA. Before joining NVIDIA in 2016, he was a principal research scientist with Mitsubishi Electric Research Labs. His object pose estimation system was awarded one of hundred most innovative technology products by the R&D magazine, in 2014. In CVPR 2018, he won the first place in both the Domain Adaptation for Semantic Segmentation Competition in the WAD challenge and the Optical Flow Competition in the Robust Vision Challenge. His research interests include generative models for image generation and understanding. His goal is to enable machines superhuman-like imagination capabilities.



Brucek Khailany (Senior Member, IEEE) received the BSE degrees in electrical and computer engineering from the University of Michigan, Ann Arbor, MI, USA and the PhD degree in electrical engineering from Stanford University, Stanford, CA, USA. He joined NVIDIA, in 2009 and is the director of the ASIC and VLSI Research group. He leads research into innovative design methodologies for IC development, ML and GPU assisted EDA, and energy efficient ML accelerators. More than 10 years with NVIDIA, he has contributed to many projects in

research and product groups spanning computer architecture and VLSI design. Previously, he was a co-founder and principal architect with Stream Processors, Inc, where he led R&D related to parallel processor architectures. With Stanford University, he led the VLSI implementation of the Imagine processor, which introduced the concepts of stream processing and partitioned register organizations.



William J. Dally (Fellow, IEEE) received the PhD degree in computer science from the California Institute of Technology, Pasadena, CA, USA, in 1986. He is the chief scientist and senior vice president of research with NVIDIA Corporation, Santa Clara, CA, USA, and an adjunct professor and former chair of computer science with Stanford University, Stanford, CA, USA. His research interests include domain-specific accelerators, parallel computer architectures, interconnection networks, and high-speed signaling circuits. He is

a member of the National Academy of Engineering, and a Fellow of ACM, and the American Academy of Arts and Sciences.



Anima Anandkumar (Fellow, IEEE) received the BTech degree from the Indian Institute of Technology Madras, India and the PhD degree from Cornell University. She did her postdoctoral research with MIT and assistant professorship with the University of California Irvine. She is a tenured professor with Caltech, and senior director of AI research with NVIDIA. She is passionate about designing principled AI algorithms and applying them to interdisciplinary domains. She has received several honors such as the IEEE fellowship, Alfred. P. Sloan Fellowship, NSF Career Award, and Faculty Fellowships from Microsoft, Google, Facebook, and Adobe. She is part of the World Economic Forum's Expert Network.

NSF Career Award, and Faculty Fellowships from Microsoft, Google, Facebook, and Adobe. She is part of the World Economic Forum's Expert Network.

▷ **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**